

PACE-Seg: When Does Input-Adaptive Semantic Segmentation Pay Off on Edge Accelerators?

Dng Nguyen^{a,*}

^a[DEPARTMENT, INSTITUTION], [STREET ADDRESS], [CITY], [POSTCODE], [COUNTRY]

Abstract

Input-adaptive inference promises to spend computation only where an image needs it, but on an edge accelerator an adaptive pipeline also pays for probing, deciding, and switching engines. We measure when this trade-off favours adaptivity, using PACE-Seg, an elastic semantic-segmentation model whose four capacities are exported as separately compiled static TensorRT and Hailo-8 engines, and a family of budget-aware routers driven by one entropy probe. Every candidate, every complete route, and every static deployment is timed in the same harness on edge devices, with Cityscapes and four ACDC conditions. The capacities span $110.2\times$ in FLOPs but only $12.3\text{--}18.1\times$ in measured mean latency, so a FLOPs-proportional proxy does not reproduce measured budget choices. Shared elastic training costs the largest capacity 0.9 mIoU points on Cityscapes and 1.6–2.8 points on ACDC relative to the same architecture trained alone. Among routers, the hard measured budget and operating-point tuning matter: routing under a hard budget outperforms rank escalation by 2.87 points on fair cells (95% CI 2.52–3.26), whereas candidate-specific calibration adds nothing over a tuned three-threshold scalar router (-0.03 points, CI -0.11 to 0.09). When each static model is charged its own measured latency, static deployment outperforms routing by 2.6 points on a TensorRT GPU and 7.4 points on a Hailo-8, because probing and switching add 0.7–2.7 ms and 38–47 ms per frame. A break-even analysis shows that routing pays off only under mean-cost budgets and only if this overhead falls below about 1.3 and 4.3 ms; adaptive segmentation studies should therefore report a static baseline timed in the same harness.

Keywords: semantic segmentation, dynamic inference, elastic neural networks, edge acceleration, hardware-aware deployment, latency measurement

1. Introduction

A vehicle or mobile robot that segments its surroundings on an embedded accelerator must return a label map within a per-frame latency budget set by its control loop. Two things make that budget hard to honour with a single network. First, visual difficulty varies from frame to frame: the same 19 semantic classes appear in clear daylight, fog, rain, snow, and at night (Fig. 1a), and a model that is sufficient for a clear frame can fail on a dark one. Second, the cost of a given network depends on the accelerator that runs it: a capacity that fits the budget on one device may not fit on another (Fig. 1b). A model chosen offline must therefore be sized for the hardest condition on the slowest device, which wastes capacity on easy frames, or it must accept budget violations.

Elastic networks offer several capacities from one set of trained weights [1, 2], but they leave open which capacity a given image warrants under a given device budget. Answering that question at run time usually involves two shortcuts. Compute counts such as FLOPs stand in for device latency, and a single confidence score stands in for

difficulty without asking which alternative capacity would actually reduce the error. Both shortcuts can be replaced by a measured or calibrated quantity, but a third cost is usually left out: the adaptive pipeline itself must probe, decide, and switch engines, which a static deployment never does (Fig. 1c). This paper asks when, once all three costs are measured on real accelerators, input-adaptive segmentation actually beats deploying one static model.

We study this question with PACE-Seg, a test bed in which every ingredient can be switched off and every cost is measured. A compiler-safe elastic model provides four capacities, tiny to large, each exported as an independent static graph. Each target device contributes a measured table of complete inference-route costs, including the probe and dispatch overhead, and of static deployment costs, timed in the same harness. For each image, the tiny candidate produces a segmentation and a mean-entropy score; a family of routers maps that score to a capacity under the budget, from rank escalation through a tuned threshold rule to candidate-specific calibrated risk (Fig. 2).

We evaluate the design through three research questions on Cityscapes, four ACDC conditions, and four edge devices (Table 1). RQ1 asks whether a FLOPs-proportional proxy reproduces measured hardware fea-

*Corresponding author.

Email address: [CORRESPONDING-AUTHOR-EMAIL] (Dng Nguyen)

sibility; it does not, because a $110.2\times$ compute range becomes only a $12.3\text{--}18.1\times$ latency range. RQ2 asks whether shared elastic training degrades quality; it does, moderately: the largest capacity is 0.9 mIoU points below the same architecture trained alone on Cityscapes and 1.6–2.8 points below on the ACDC conditions. RQ3 asks which routing ingredients matter and whether routing beats static deployment. With measured route costs on a TensorRT GPU and a Hailo-8 accelerator, the hard budget and fit-half operating-point tuning account for the gain over rank escalation, and a tuned three-threshold scalar router matches candidate-specific calibration. Against static deployment charged its own measured latency, routing loses on both accelerators, and a break-even analysis quantifies how small the routing overhead would have to become.

The contributions are:

1. a measurement methodology that times separately compiled static candidates, complete inference routes, and static deployments with one harness and one timer boundary on a TensorRT GPU and a Hailo-8, and shows that a FLOPs-proportional proxy misstates feasibility;
2. a same-harness comparison of adaptive routing with static deployment and a break-even analysis of the routing overhead, which identify the conditions under which adaptive segmentation pays off on these accelerators;
3. an ablation of budget-aware routing that separates the hard budget, operating-point tuning, and candidate-specific calibration, with image-level and nested bootstrap intervals, showing that candidate-specific calibration is not necessary with a single scalar probe;
4. compiled-engine accuracy for TensorRT FP16 and Hailo-8 INT8 and an on-device replay that confirm the conclusions with the deployed engines.

Section 2 positions the work. Section 3 follows one image through PACE-Seg, Section 4 describes the data, hardware, and evaluation protocol, and Section 5 answers RQ1–RQ3. Section 6 discusses when routing pays off, what drives the gain among routers, and what the study does not establish.

2. Related work

2.1. Efficient semantic segmentation

Fully convolutional networks established dense end-to-end prediction [3], and pyramid pooling and atrous encoder-decoders raised accuracy at substantial computational cost [4, 5]. Real-time designs reduce that cost in several ways. ENet and ERFNet use early downsampling and factorized convolutions [6, 7], ICNet cascades low- and high-resolution branches [8], and Fast-SCNN shares a learning-to-downsample module between a global feature

Table 1: Study setting at a glance. Candidate-only latency is measured on four devices; complete routed latency, including probing and decision overhead, and static latency are measured on the two primary accelerator backends.

Component	Evaluated setting
Visual domains	Cityscapes clean and ACDC fog, night, rain, and snow
Prediction task	Semantic segmentation over the 19 Cityscapes train classes
Elastic choices	Tiny, small, medium, and large static capacities
Candidate-cost devices	E1, E2, E3, and E5
Complete-route backends	E1 Hailo-8 and E3 TensorRT/CUDA
Router protocol	Alternating fit/held-out validation halves; three training seeds

extractor and a high-resolution path [9]. Fast-SCNN serves as our matched-capacity comparator because it occupies a similar sub-million-parameter regime with the depthwise-separable operators that also underlie mobile classification networks [10, 11].

Current real-time models are dominated by two-branch and multi-resolution designs. BiSeNet and BiSeNetV2 separate spatial detail from semantic context [12, 13], STDC replaces the extra spatial path with detail-guided training [14], DDRNet maintains bilateral dual-resolution fusion [15], and PIDNet adds a boundary branch inspired by PID control [16]. SegFormer and OffSeg improve the accuracy–efficiency frontier through efficient attention and offset-based feature alignment [17, 18]. Despite their different mechanisms, these models share one deployment assumption: after compilation, every image passes through the same operating point. PACE-Seg is complementary to this line. Any family of compilable candidates could replace its elastic encoder–decoder; we keep one family fixed so that routing effects are not confounded with architectural differences.

2.2. Elastic and input-adaptive networks

Slimmable networks train shared weights at several preset widths with switchable batch normalization, and US-Net extends this to arbitrary widths with the sandwich rule and in-place distillation [19, 1]. Once-for-All trains a broader depth–width–resolution–kernel space and extracts a specialized subnetwork for each target under a latency predictor [2]. SlimSeg brings slimmable training to segmentation with stepwise distillation [20], and recent width-adaptive ConvNeXt models target deployment of one network across devices with different resource limits [21]. PACE-Seg adopts the sandwich rule and in-place distillation, but freezes each capacity into a separate static graph instead of switching widths inside one dynamic model, which keeps the engines compatible

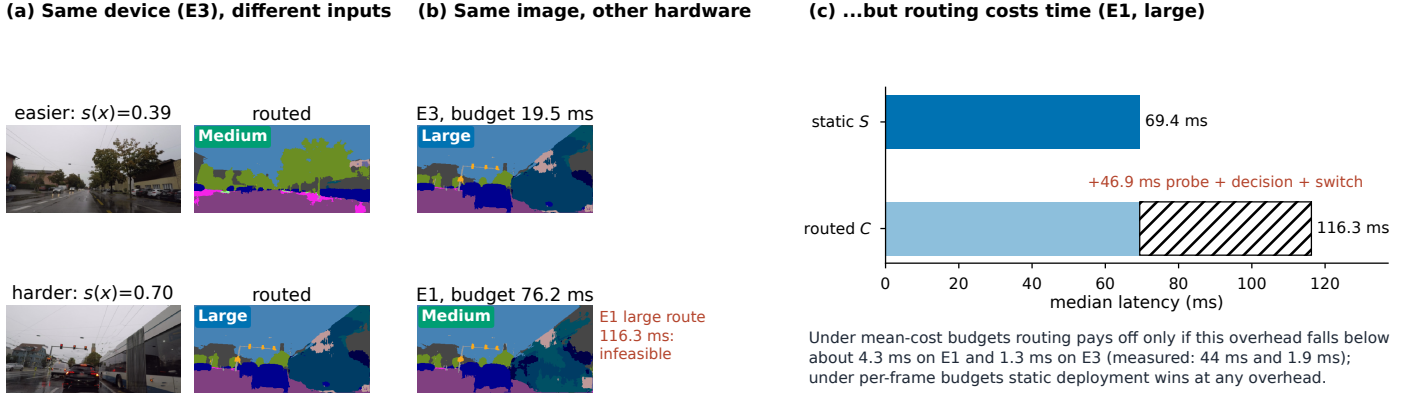


Figure 1: Adaptive capacity selection on an edge accelerator, and its price. (a) On one device (E3, budget of the large route), an easier and a harder held-out ACDC rain image receive different capacities from policy D. (b) The same harder image receives large on E3 but medium on the Hailo-8 E1, where the large route exceeds the budget. (c) On E1 the route to large costs 46.9 ms more than deploying large statically, because the probe, decision, and engine switch must run first; overheads of this size erase the adaptive gain (Section 5.3). Images are the frozen ‘easier’ and ‘harder’ rain selections of Section 5.3; decisions are those of Run A.

with accelerators such as Hailo-8 that do not execute data-dependent control flow.

A related line adapts computation to each input. Early-exit classifiers such as BranchyNet and MSDNet stop at an intermediate predictor once confidence is sufficient [22, 23], and SkipNet and BlockDrop learn per-input layer or block execution [24, 25]; Han et al. survey these sample-, spatial-, and temporal-wise variants [26]. For dense prediction, Dynamic Routing selects scale-transform paths with learned gates [27], Anytime Dense Prediction refines its output through confidence-adaptive exits [28], and MESS chooses among the exits of a multi-exit segmentation network [29]. Most of these methods express cost through FLOPs or a soft training penalty and execute their adaptivity inside one graph. PACE-Seg moves the decision outside the graph and constrains it by measured latency, at the price of coarser granularity: it chooses among four engines rather than among layers or exits.

2.3. Hardware-aware design and latency modelling

MnasNet made measured on-device latency a search objective [30], and ProxylessNAS and FBNet searched directly on target hardware or through latency lookup tables [31, 32]. FasterSeg, AutoSegEdge, multi-objective edge NAS, RealtimeSeg, and HARD bring platform cost into segmentation architecture design [33, 34, 35, 36, 37]. HW-NAS-Bench documented that FLOPs and measured latency diverge across devices [38], and nn-Meter predicts kernel-level latency for unseen models [39]. All of these use hardware cost to choose one architecture before deployment. PACE-Seg uses cost at run time to restrict a fixed candidate set per device; because the set is small, every complete route can be measured directly and no latency predictor is needed. RQ1 accordingly evaluates only a proportional FLOPs proxy and makes no claim about learned predictors.

2.4. Uncertainty and calibration under adverse conditions

Monte Carlo dropout, deep ensembles, and aleatoric-epistemic decompositions estimate predictive uncertainty [40, 41, 42], but require multiple passes or extra heads that are costly on edge accelerators. Modern networks are often miscalibrated [43], and monotone post-hoc mappings such as histogram binning and isotonic regression are standard corrections [44]. Selective prediction asks whether a score ranks errors correctly and summarizes this with risk-coverage curves and AURC [45]; conformal risk control adds distribution-free guarantees under exchangeability [46]. In segmentation, calibration degrades under domain shift and differs across model capacities [47, 48]. Foggy Cityscapes and Dark Zurich target fog and night [49, 50], and ACDC provides real fog, night, rain, and snow images with Cityscapes-compatible labels [51]. PACE-Seg uses a single-pass entropy score and a separate binned monotone calibrator per candidate; its estimates support risk-aware selection but provide no formal risk guarantee.

2.5. Positioning

Table 2 compares the closest methods by how they construct candidates, when they select, and how they represent cost and risk. Most input-adaptive methods report savings in FLOPs or against static baselines with idealized costs. What the present experiments establish is narrower and deployment-oriented: separately compiled static candidates, measured complete inference routes on TensorRT and Hailo-8, a static comparison timed in the same harness, compiled-engine validation, and the break-even overhead at which adaptivity would pay off. Candidate-specific calibrated risk is one of the evaluated routers, not a component the results show to be necessary.

Table 2: Mechanism-level positioning relative to representative primary work. Entries summarize the stated operating mechanism rather than rank methods or claim exhaustive coverage. “Complete route” includes the probe, risk computation, policy decision, backend dispatch or activation, and any additional candidate inference.

Method	Candidates	Selection	Cost	Risk	Budget; deployed form
Once-for-All [2]	Elastic subnetworks	Design	Latency predictor	None	Design constraint; static subnet
SlimSeg [20]	Slimmable widths	Configured	Efficiency setting	None	No hard measured budget; static width
MESS [29]	Multi-exit network	Per image	Exit latency	Exit confidence	No complete-route constraint; multi-exit graph
Dynamic Routing [27]	Multi-scale paths	Per image	Compute penalty	Learned gates	No measured accelerator constraint; dynamic paths
AutoSegEdge [34]	Searched architecture	Design	FLOPs + device latency	None	Search constraint; static architecture
PACE-Seg	Four elastic capacities	Per image	Measured complete route	Candidate-specific error	Hard measured budget; static compiled engines

3. PACE-Seg

Figure 2 summarizes how PACE-Seg processes one image. Two quantities are computed independently. Visual risk is estimated per image from a cheap probe, and hardware cost is measured once per device. They meet only in the policy, which selects one static engine. This section describes the candidate family that the policy chooses from (Section 3.1), the two estimated quantities (Sections 3.2 and 3.3), the policy itself (Section 3.4), and a complete worked decision (Section 3.5).

3.1. Compiler-safe elastic candidate family

The four candidates, tiny, small, medium, and large, are extracted from one shared encoder-decoder by jointly varying three axes: width multiplier (0.25, 0.50, 0.75, 1.00), repeated-block depth (2, 3, 4, 6), and input resolution (384×192 , 512×256 , 768×384 , 1024×512 , width $\times$ height, matching the 2:1 aspect of Cityscapes and ACDC). The candidates share convolutional weights through active-channel slicing, while each keeps its own batch-normalization statistics, following slimmable training practice [19]; activation statistics of a 0.25-width, 384×192 path differ substantially from those of the full-width, 1024×512 path. The network uses only standard operators and fixed-factor resizing, so each candidate exports as a fixed-shape ONNX graph and compiles to a static TensorRT or Hailo-8 engine (Fig. 3). Routing therefore chooses an engine before execution and never introduces data-dependent control flow into a graph.

All candidates are trained together with the sandwich rule [1]. Each step samples the smallest level, the largest level, and one randomly chosen intermediate level; every sampled level receives the batch resized to its own resolution. Each level is supervised with a boundary-aware cross-entropy, $\mathcal{L}_{\text{seg}} = \text{CE} + \lambda_b \text{CE}_w$, where CE_w reweights label-boundary pixels by three and $\lambda_b = 1$. Every level except the largest additionally receives in-place distillation [52] from the detached, resized logits of the largest

level, $\alpha T^2 \text{KL}(p_{\text{large}}^T \| p_{\ell}^T)$ over non-ignore pixels, with $\alpha = 0.5$ and $T = 1$; the step loss sums all sampled levels. Training runs for 100,000 steps with batch size 8 and AdamW (initial learning rate 3×10^{-4} , weight decay 10^{-4}), 500 warm-up steps followed by cosine decay, gradient clipping at 5.0, and FP16 mixed precision. The Cityscapes and ACDC training sets are concatenated without reweighting, and augmentation applies random scaling in $[0.75, 1.25]$, padding and cropping, horizontal flipping with probability 0.5, and colour jitter of 0.3 in brightness, contrast, and saturation.

3.2. Measured route cost

For a target device h and candidate ℓ , the policy uses the measured complete inference-route cost $C_h(\ell)$, the time from handing a preprocessed host tensor to the run-time until the selected candidate’s output is synchronized; image decoding, resizing, and normalization are excluded. A route always begins with the tiny probe, so $C_h(\ell)$ includes tiny inference, entropy, calibrator and policy evaluation, backend dispatch or activation, and, unless tiny itself is selected, inference of the chosen candidate. When tiny is selected, its probe prediction is returned directly. Because cost enters only after training, deploying to a new device requires compiling the four graphs and measuring their four routes, with no change to weights or calibrators. Section 5.1 shows why this measurement cannot be replaced by FLOPs. For comparison with static deployment we also measure the static cost $S_h(\ell)$ of running candidate ℓ alone, without the probe, in the same harness.

3.3. Candidate-specific visual risk

Let the tiny candidate produce logits $z(x)$ for image x , with softmax probabilities $p_{u,c}$ at pixel u for class c . The probe score is mean pixel entropy,

$$s(x) = \frac{1}{|\Omega|} \sum_{u \in \Omega} - \sum_{c=1}^C p_{u,c} \log \max(p_{u,c}, 10^{-12}), \quad (1)$$

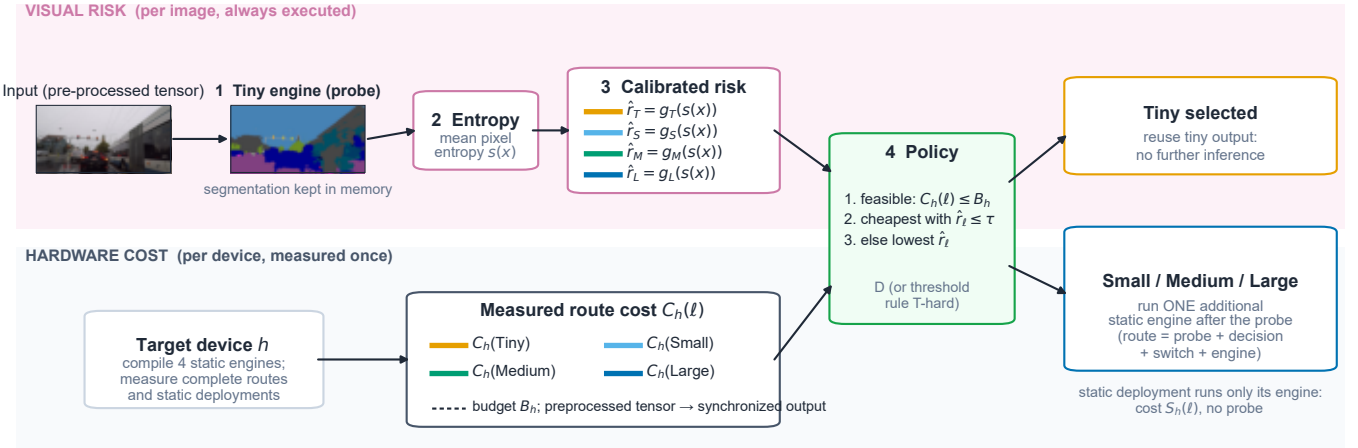
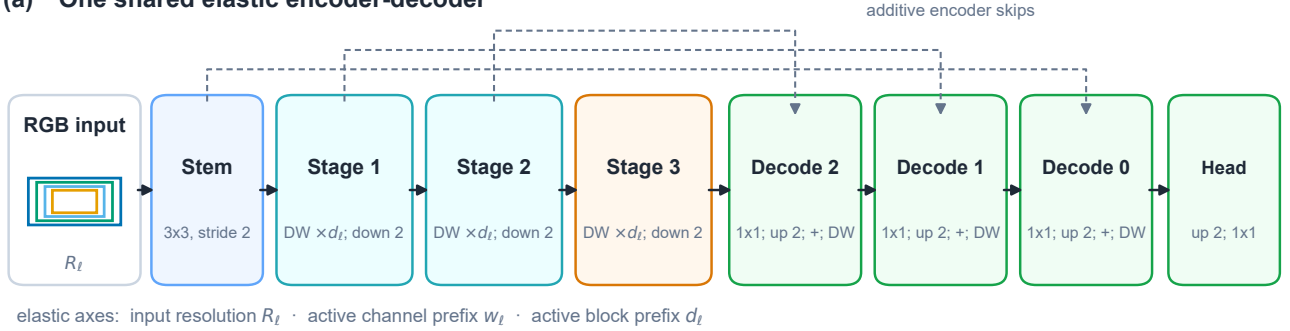


Figure 2: PACE-Seg inference. The tiny probe always runs first: it produces a segmentation and its mean entropy $s(x)$, which candidate-specific calibrators g_ℓ map to expected errors $\hat{r}_\ell$ (top lane). Hardware cost is measured once per device as the complete inference-route cost $C_h(\ell)$, from a preprocessed host tensor to synchronized output, together with the budget B_h (bottom lane). If the policy selects tiny, the probe output is returned with no further inference; otherwise one additional static engine runs after the probe. A static deployment runs only its own engine and pays $S_h(\ell)$. Fig. 4 gives a numerical instance.

(a) One shared elastic encoder-decoder



(b) Four related static candidates from one trained parameter family

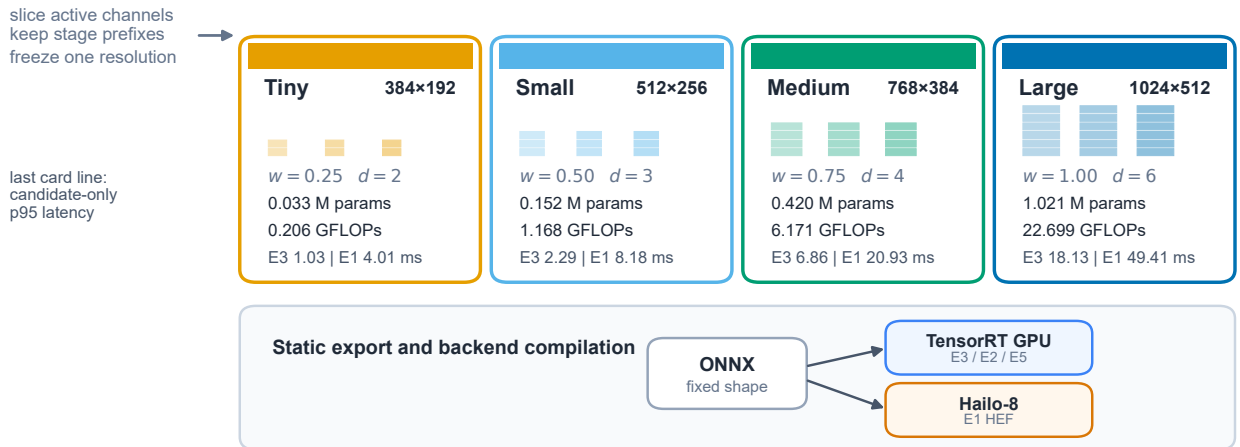


Figure 3: Elastic architecture and static extraction. (a) One shared encoder-decoder: a stem, three downsampling depthwise-separable encoder stages, three top-down decoder stages with additive encoder skips, and a 19-class head; input resolution R_ℓ , active channel prefix w_ℓ , and active block prefix d_ℓ define elasticity. (b) Tiny through large freeze those axes into four static graphs, exported through fixed-shape ONNX to TensorRT GPU or Hailo-8. Card values are input size (width×height), parameters, GFLOPs, and candidate-only p95 latency on E3 and E1; complete route costs are measured separately (Table 6).

over all output pixels Ω , with $C = 19$. For each candidate ℓ , a monotone calibrator g_ℓ maps the shared score to that candidate’s expected per-image pixel error,

$$\hat{r}_\ell(x) = g_\ell(s(x)). \quad (2)$$

Each g_ℓ is fitted on the fit half of the data only: fit-half scores are divided into ten equal-count bins, the mean observed error of candidate ℓ is computed in each bin over non-ignore pixels, and a cumulative-maximum pass enforces monotonicity. The primary configuration fits one calibrator set per visual condition; a pooled variant fits one set on all five fit halves and removes the need to know the condition at run time.

Three properties motivate this design. The probe costs no extra network, because the tiny engine is itself a deployable route and its entropy needs only one softmax and one reduction, a cost already contained in every measured route. Multi-pass estimators such as Monte Carlo dropout or ensembles [40, 41] would multiply that cost. A single score cannot rank candidates by itself, since the same image can be hard for tiny yet easy for large; fitting one calibrator per candidate converts the shared score into a vector of expected errors, and the differences within that vector drive the decision. Finally, binned monotone calibration [44] has few parameters per candidate and cannot reverse the ordering implied by the score, which limits overfitting on fit halves of 50–250 images per split.

3.4. Hardware-cost-conditioned policy

Given the risk vector $\hat{r}_\ell(x)$, the cost table $C_h(\ell)$, a budget B_h , and a risk target τ , policy D first forms the feasible set

$$\mathcal{S}_h = \{\ell : C_h(\ell) \leq B_h\}. \quad (3)$$

If some feasible candidate satisfies $\hat{r}_\ell(x) \leq \tau$, D selects the cheapest such candidate; otherwise it selects the feasible candidate with the lowest predicted error, breaking ties by cost. If $\mathcal{S}_h$ is empty, that is, $B_h < C_h(\text{tiny})$, D returns the tiny probe output, the cheapest possible route. With four candidates the decision costs four table lookups and four comparisons, independent of image resolution. A new device changes only C_h ; a new visual domain changes only g_ℓ .

The main comparator, policy A, uses the same calibrated probe score $\hat{r}_{\text{tiny}}(x)$ but escalates by latency rank: it selects the cheapest candidate if the score is at most τ and otherwise the candidate at latency rank $\lceil \hat{r}_{\text{tiny}}(x)/\tau \rceil$, counting the cheapest as rank 0 and capping at large. A has neither candidate-specific risk nor a hard budget. To separate these two ingredients, A-hard applies D’s feasibility mask to A: whenever A’s choice exceeds B_h , it is replaced by the most expensive feasible candidate. A-hard therefore has a hard budget but no candidate-specific risk. T-hard asks whether candidate-specific calibration is needed at all: it applies three monotone thresholds directly to $s(x)$, fitted on the fit half to maximize fit-half mIoU under the budget, with the same feasibility mask. Table 3

lists all evaluated policies and the information each may access. Two static references run one capacity without the probe: the best feasible static candidate, the largest whose cost fits the budget, and, for mean-cost budgets, a random mixture of two adjacent static candidates. A per-image ground-truth pixel-error oracle, which selects for each image the feasible candidate with the lowest observed pixel error, serves as a non-deployable reference; because it optimizes per-image pixel error rather than dataset-level mIoU, it is not a formal upper bound.

3.5. A worked decision

Figure 4 follows one held-out ACDC rain image of Run A through policy D. The tiny probe yields $s(x) = 0.704$ nats. The four calibrators turn this score into expected errors of 0.189, 0.155, 0.130, and 0.110 for tiny, small, medium, and large; none meets the risk target $\tau = 0.062$ chosen on the fit half, so D falls back to the feasible candidate with the lowest predicted error. On E3, with a budget equal to the large route (19.5 ms), every route is feasible and D runs large. On E1, with a budget equal to the medium route (76.2 ms), the large route (116.3 ms) is infeasible and D runs medium. The image and the risk vector are identical; only the measured cost table changes the action. The example shows the property that distinguishes D from rank escalation: the decision uses the full risk vector, and infeasible candidates are removed before risk is consulted.

4. Experimental setup

4.1. Data and fit/held-out split

Cityscapes provides 2,975 training and 500 validation images [53], and ACDC provides 1,600 training and 406 validation images across fog, night, rain, and snow [51]; both use the 19 Cityscapes training classes. All models are trained on the training sets at the native 2:1 aspect ratio. Each validation split is then divided by alternating index (Table 4): even-index images form the fit half, used for calibration and operating-point selection, and odd-index images form the held-out half, used only for evaluation. The study does not use an external test server.

4.2. Models and training runs

Three elastic supernets are trained with seeds 0–2 (Runs A–C) under the recipe of Section 3.1, and every router experiment is repeated on all three. For RQ2, two references are trained with the same recipe and three seeds each: Fast-SCNN [9], and the large PACE-Seg architecture trained alone, without weight sharing or distillation (PACE-Large). Checkpoint and configuration hashes are listed in the supplement.

Table 3: Routing policies and information access. “Per condition” means that the deployment domain is configured before inference; pooled D removes that requirement. The oracle uses held-out ground-truth pixel error and is not deployable.

Policy	Candidate risk	Hard budget	Calibrator	Deployable	Purpose
Best feasible static	No	Yes	None	Yes	Non-adaptive reference
Static mixture	No	Mean cost	None	Yes	Reference under mean-cost budgets
Policy A: rank escalation	No	No	Per condition	Yes	Rank-based comparator
Policy A-hard	No	Yes	Per condition	Yes	Isolates the hard budget
Policy T-hard	No	Yes	Thresholds on $s(x)$	Yes	Isolates candidate-specific calibration
Policy D: configured	Yes	Yes	Per condition	Yes	Primary method
Policy D: pooled	Yes	Yes	Pooled	Yes	Condition-agnostic check
GT pixel-error oracle	GT error	Yes	None	No	Reference only

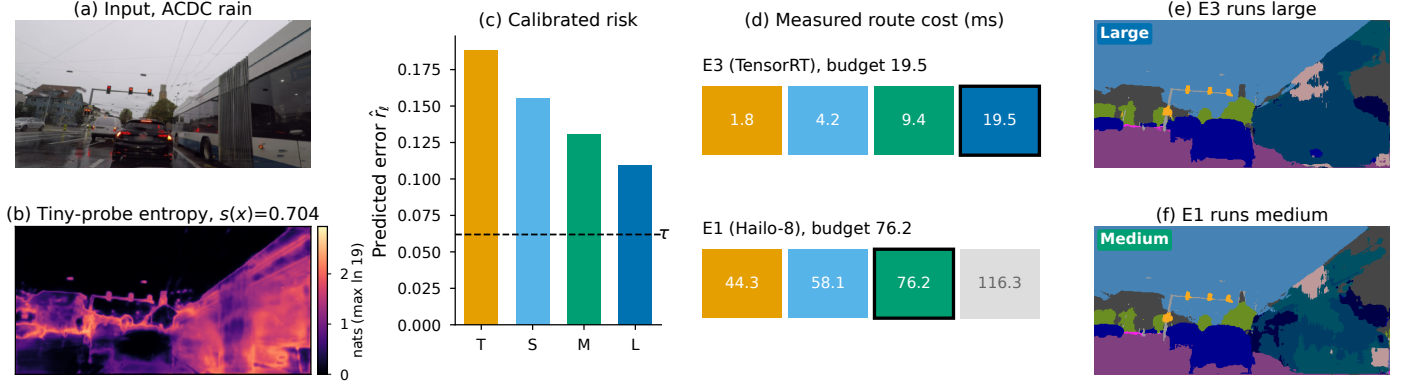


Figure 4: One complete policy-D decision on a held-out ACDC rain image. (a) Input. (b) Per-pixel entropy of the tiny probe and its mean $s(x)$. (c) Candidate-specific calibrators map $s(x)$ to four predicted errors; the dashed line is the risk target τ . (d) Measured median route costs on E3 and E1 under the stated budgets; infeasible routes are grey and hatched, and the selected route is outlined. (e, f) The capacity run on each device. The example illustrates the policy and is not an additional evaluation trial.

Table 4: Dataset composition and alternating-index router split. Fit and held-out counts partition each validation subset exactly.

Dataset/condition	Train	Validation	Fit	Held-out
Citiescapes clean	2,975	500	250	250
ACDC fog	400	100	50	50
ACDC night	400	106	53	53
ACDC rain	400	100	50	50
ACDC snow	400	100	50	50

Table 5: Hardware and compiler stacks used for latency measurement.

ID	Hardware	Runtime and system versions
E1	Raspberry Pi 5 + Hailo-8 M.2	HailoRT 4.23.0; Dataflow Compiler 3.34.0; Model Zoo 2.19.0
E2	Jetson Xavier NX	L4T R35.4.1; CUDA 11.4.19; cuDNN 8.6.0.166; TensorRT 8.5.2.2
E3	Jetson AGX Xavier	L4T R35.6.4; CUDA 11.4.19; TensorRT 8.5.2.2
E5	Jetson Orin Nano Super	L4T R36.5.0; CUDA 12.6.68; cuDNN 9.3.0; TensorRT 10.3.0.30

4.3. Hardware and latency measurement

Table 5 lists the four devices. NVIDIA devices run TensorRT FP16 engines built from the fixed-shape ONNX graphs, and E1 runs INT8 HEF files compiled from the same graphs with the Hailo Dataflow Compiler and quantized with 200 real fit-half images. All four candidates execute on both primary paths. The Jetson DLA is excluded because decoder layers fall back to the GPU, which would make the measured route a mixed-device path.

Three kinds of latency are measured. Candidate-only latency, used for RQ1, times each candidate in isolation at batch size one on all four devices with the `trtexec/hailortcli` protocol; RQ1 uses the mean, and the cards of Fig. 3 report p95. Complete route cost $C_h(\ell)$, used by the router, and static cost $S_h(\ell)$, used for the static references, are measured on E1 and E3 in the same process, with the same en-

gines, inputs, and timer boundary. The timer starts when a preprocessed host tensor is handed to the runtime and stops after the returned prediction is synchronized; image decoding, resizing, and normalization are outside the timer. On E3 all four TensorRT execution contexts stay resident and the route covers tiny inference, entropy, policy evaluation, dispatch, and any additional inference. The primary E1 path uses explicit Hailo network-group activation with FLOAT32 output streams, and its route includes the activation of the selected engine; the HailoRT model scheduler and UINT8 output streams are reported as sensitivity results. Each configuration uses 50 warm-up and 500 timed iterations, with Jetson power mode recorded and temperatures logged before and after each run. Table 6 reports the medians, p95, and p99 used in replay. Cold-start costs are excluded because the two runtimes have different reload

Table 6: Warm complete-route cost and static cost in ms (median / p95 / p99 over 500 timed iterations, same harness). A route includes the tiny probe, risk and policy computation, dispatch or activation, and any additional candidate inference; a static deployment runs only its own engine. The median is the primary cost table; p95 and p99 tables are used for the sensitivity analysis in Section 5.3.

Route	E3 TensorRT/CUDA	E1 Hailo-8
tiny $\rightarrow$ tiny	1.84 / 1.91 / 1.94	44.26 / 45.28 / 45.42
tiny $\rightarrow$ small	4.24 / 4.36 / 4.42	58.11 / 59.56 / 60.00
tiny $\rightarrow$ medium	9.44 / 9.78 / 9.87	76.18 / 78.23 / 80.16
tiny $\rightarrow$ large	19.50 / 19.59 / 19.63	116.26 / 118.02 / 118.93
static tiny	1.18 / 1.21 / 1.24	6.27 / 6.57 / 6.61
static small	2.56 / 2.71 / 2.76	12.07 / 12.67 / 12.82
static medium	7.11 / 7.42 / 7.51	30.91 / 31.82 / 32.07
static large	16.76 / 16.92 / 17.26	69.39 / 70.51 / 70.98

semantics.

Accuracy and latency are combined in an offline per-image replay with measured costs. Every candidate prediction is computed once per image from the trained PyTorch models and cached, and each routing decision is scored against these cached predictions while its cost is taken from the measured table. Section 5.4 checks this replay against the compiled TensorRT and Hailo-8 engines with the trained weights.

4.4. Evaluation protocol

Segmentation quality is dataset-level mIoU. For a routed policy, the confusion matrices of the selected candidates are accumulated over the held-out images and mIoU is computed once. The unit of analysis is an operating cell, defined by training run, backend cost table, split, and budget; the 120 cells reuse images and predictions and are therefore correlated, and we report descriptive win/tie/loss counts alongside bootstrap intervals.

Operating points are chosen on the fit half by a fixed rule. Five candidate risk targets τ are set to the 10th, 25th, 50th, 75th, and 90th percentiles of fit-half observed error, macro-aggregated across splits; one such grid is generated per run and shared by the E1 and E3 cost tables. For each budget and policy, the selected target maximizes fit-half mIoU among targets whose mean fit-half cost meets the budget, with lower cost as the tie-break. Only then are held-out labels read, to compute routed mIoU and budget violations. Calibrators, targets, thresholds, and operating points never see held-out labels, the entropy feature is identical at fitting and deployment, and the pooled variant combines only fit halves.

A cell is violating if at least one held-out image is routed to a candidate whose tabulated route cost exceeds the budget. This is a route-selection criterion: it guarantees that the selected route’s median (or, in the sensitivity analysis, p95 or p99) cost fits the budget, not that every individual frame does. Because D removes infeasible candidates before choosing, its zero violation count is partly structural, whereas A can violate a budget whenever its

threshold escalates an image to an expensive route. Comparisons with A therefore use fair cells, those in which A has no held-out violation; comparisons with A-hard, T-hard, and the static references use all 120 cells, since none of them violates. Router comparisons use the four route costs as budgets; comparisons with static deployment use the union of the route and static costs of each device. Quality differences are reported in mIoU percentage points.

Uncertainty is estimated with an image-level bootstrap: in each of 1,000 replicates, the held-out images of every split are resampled with replacement, the same resample is applied to every run, backend, and budget of that split, routed mIoU is recomputed, and the macro-averaged difference is recorded. We report percentile 95% intervals. A nested bootstrap (200 replicates) additionally resamples the fit halves and refits calibrators, risk grid, and operating points in every replicate. Three further analyses address robustness. The cost-statistic sensitivity rebuilds both the cost table and the budget grid from the p95 or p99 costs. The leave-one-condition-out (LOCO) analysis refits the calibrators, the risk-target grid, and the operating points on the fit halves of the other four conditions only and evaluates on the held-out half of the remaining condition. Finally, a break-even analysis scales the measured routing overhead $C_h(\ell) - S_h(\ell)$ by a factor $\alpha \in [0, 1]$ and repeats the comparison with static deployment.

5. Results

5.1. RQ1: Does a FLOPs-proportional proxy reproduce measured feasibility?

No. The four candidates require 0.206, 1.168, 6.171, and 22.699 GFLOPs, a $110.2\times$ range from tiny to large, but their measured mean latency grows by only $12.3\text{--}18.1\times$, depending on the device: $12.3\times$ on the Hailo-8 (4.0 to 49.3 ms), $17.2\times$ on E3 (1.0 to 17.2 ms), $17.5\times$ on E5 (1.0 to 18.0 ms), and $18.1\times$ on E2 (1.6 to 29.5 ms) (Fig. 5). The dashed lines in Fig. 5 show the latency that a proxy proportional to FLOPs, anchored at each device’s tiny candidate, would predict; it overestimates the cost of every larger candidate, and that of large by $6\text{--}9\times$ (110 ms instead of 17.2 ms on E3, 441 ms instead of 49.3 ms on E1). The small candidates are memory- and launch-bound, so their latency falls far less than their compute, and the gap differs between devices because each accelerator handles memory traffic and small kernels differently.

The mismatch changes deployment decisions. Under the illustrative 10 ms budget marked in Fig. 5, the largest feasible candidate is small on E1 and E2 and medium on E3 and E5, whereas the proxy would rule out medium on E3 and E5 and small on E1, all of which measurement shows to fit. Measuring the four routes once per device removes this error at negligible engineering cost. The result concerns a proportional proxy only; learned latency predictors may do better and are not evaluated here.

Isolated candidate latency is itself insufficient, because a deployed route also includes the probe and dispatch. On

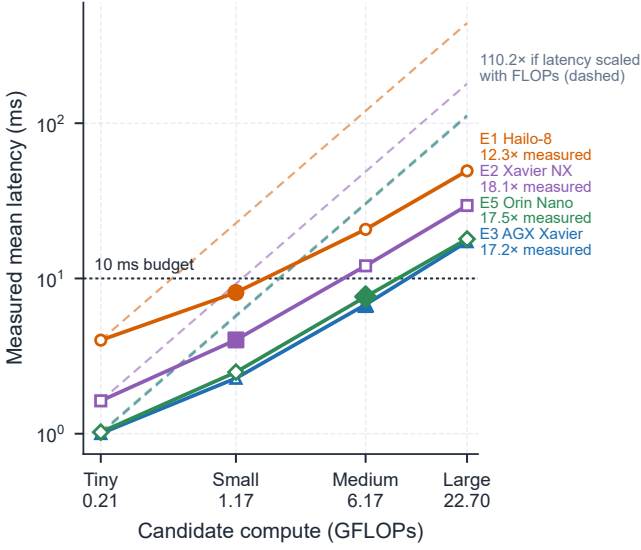


Figure 5: Measured latency versus compute. Solid lines show measured mean latency of the four candidates on each device; dashed lines in the same colour show the latency a FLOPs-proportional proxy anchored at tiny would predict. Filled markers indicate the largest candidate that fits an illustrative 10 ms budget on each device. The 110.2 $\times$ compute range becomes only 12.3–18.1 $\times$ measured latency.

E3 the tiny route grows from 1.03 ms candidate-only p95 to a 1.84 ms complete-route median, and on E1 from 4.01 ms to 44.26 ms, where network-group activation and host-side entropy dominate the short tiny inference in the primary E1 configuration. On E1 this expansion falls from 11.0 $\times$ for tiny to 2.4 $\times$ for large as candidate inference begins to dominate, whereas on E3 it stays between 1.1 $\times$ and 1.9 $\times$. A budget defined from candidate-only latency would admit routes that are infeasible in deployment, particularly on E1. This is why the router uses complete route costs (Table 6).

5.2. RQ2: Does shared elastic training preserve quality?

Only partly. Figure 6 compares the largest elastic capacity with the same architecture trained alone (PACE-Large) and with Fast-SCNN, each over three seeds. Shared training costs the large capacity 0.9 mIoU points on Cityscapes (59.0 ± 0.3 versus 60.0 ± 0.5) and 1.6–2.8 points on the ACDC conditions, 2.0 points on the ACDC mean (49.8 versus 51.8); the gap is largest at night (−2.8). Against Fast-SCNN, the elastic large capacity is 0.8 points better on Cityscapes and 0.8–1.8 points worse on night, snow, and fog, but 4.0 points worse on rain. Our study plan had set a practical band of 1.5 points relative to an independently trained model of similar size; the elastic large capacity stays within it on Cityscapes, night, and snow but not on fog or rain. In exchange, the same weights also provide the tiny, small, and medium capacities (33.8, 40.8, and 49.3 mIoU on Cityscapes).

This result matters for interpreting RQ3. A router can only choose among the candidates it is given, and the cost

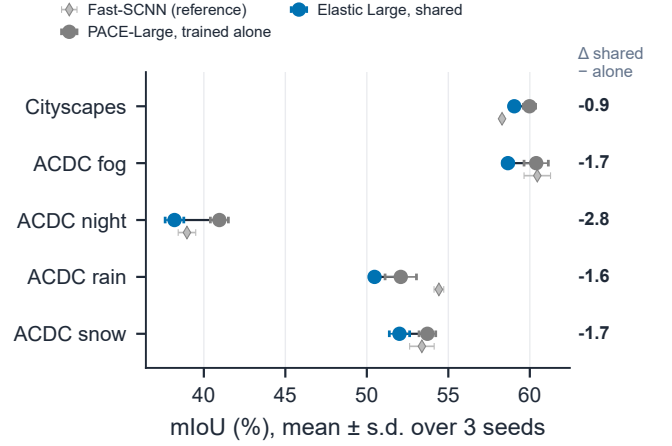


Figure 6: Weight sharing costs the largest capacity 0.9–2.8 mIoU points. Elastic Large (shared training, blue) versus the same architecture trained alone (PACE-Large, grey) per split, mean $\pm$ s.d. over three seeds; the right column gives the difference. Fast-SCNN (diamonds) is a secondary external reference.

of weight sharing grows under adverse conditions, which is exactly where a router would most want the large capacity. All routing comparisons below are made within one supernet, so they are not affected by this gap, but a deployment that needs only the large capacity would be better served by training it alone.

5.3. RQ3: What does candidate-specific routing add?

We answer RQ3 against five references: rank escalation (A), rank escalation with the same hard budget (A-hard), a three-threshold scalar router (T-hard), the best feasible static candidate, and, under mean-cost budgets, the static mixture. Table 7 summarizes all comparisons in the replay with measured costs.

Against rank escalation. Configured D outperforms or matches A in 60 of 62 fair cells, with a mean gain of +2.87 mIoU points (95% CI +2.52 to +3.26; nested +1.85 to +3.28), and it never selects a route whose tabulated cost exceeds the budget, whereas A does so in 58 of 120 cells. Pooled D, which uses one calibrator set for all conditions, records 54/6/2 and +2.90 points. Figure 7 summarizes the router ablation and the static comparisons with their intervals.

Separating the hard budget from candidate-specific risk. D differs from A in two ways, a hard budget and candidate-specific risk. A-hard adds only the first. Adding the hard budget removes all of A’s violations but improves fair-cell quality by only +0.47 points. Adding candidate-specific risk on top, which turns A-hard into D, gains a further +1.41 points (95% CI +1.19 to +1.63; nested +0.99 to +1.70) with 48 wins, 72 ties, and no losses over all 120 cells. The same gain, however, is obtained without candidate-specific calibration: T-hard, which places three

Table 7: Routing comparisons in held-out replay with measured costs (three runs $\times$ two backends $\times$ five splits $\times$ four budgets = 120 cells). Δ is the macro-averaged mIoU difference in percentage points; the 95% interval is an image-level bootstrap, and the nested interval also refits calibrators, risk grid, and operating points on resampled fit halves (Section 4.4). Comparisons with A use fair cells, in which A has no violation; all other comparisons use the full grid. LOCO refits calibrators, risk targets, and operating points without the evaluated condition. “Static, route cost” charges the static candidate the cost of the route that ends in it; “static, own cost” charges it its measured static latency (Table 6). Cells are correlated and E1/E3 share predictions. Violating cells (median / p95 / p99 tables): A 58 / 58 / 59 of 120; A-hard, T-hard, D, pooled D, static, and LOCO D 0 of 120 in every table.

Cost table	Comparison	Cells	W/T/L	Mean Δ	95% CI	Nested CI
Median	D – A	62	54/6/2	+2.87	[+2.52, +3.26]	[+1.85, +3.28]
Median	Pooled D – A	62	54/6/2	+2.90	–	–
Median	A-hard – A	62	22/38/2	+0.47	–	–
Median	D – A-hard	120	48/72/0	+1.41	[+1.19, +1.63]	[+0.99, +1.70]
Median	T-hard – A-hard	120	46/68/6	+1.44	–	–
Median	D – T-hard	120	16/80/24	−0.03	[−0.11, +0.09]	–
Median	D – best feasible static, route cost	120	0/92/28	−0.16	[−0.22, −0.10]	–
Median	D – best feasible static, own cost, E3	120	0/61/59	−2.62	[−2.69, −2.37]	–
Median	D – best feasible static, own cost, E1	120	0/21/99	−7.41	[−7.62, −6.75]	–
p95	D – A	62	54/6/2	+2.87	[+2.52, +3.26]	–
p95	D – A-hard	120	48/72/0	+1.41	[+1.19, +1.63]	–
p99	D – A	61	53/6/2	+2.83	–	–
Median, LOCO	D – A	41	29/12/0	+4.15	–	–
Median, LOCO	D – A-hard	120	48/72/0	+1.59	–	–

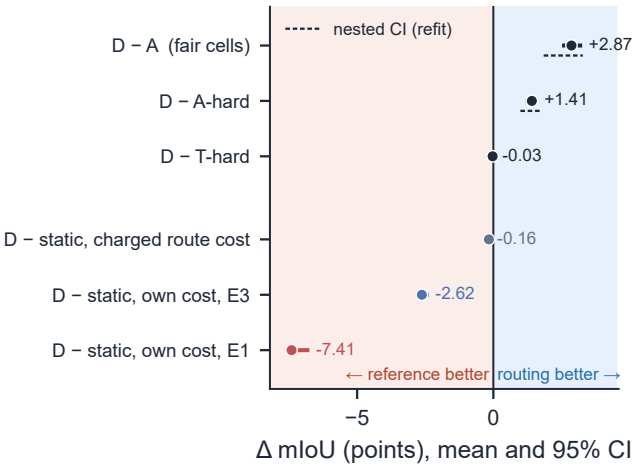


Figure 7: Routing gains come from the hard budget, not from candidate-specific calibration, and vanish against static deployment charged its own latency. Mean mIoU difference of policy D against each reference with image-level 95% bootstrap intervals (solid) and nested intervals that refit calibrators and operating points (dashed, where computed); median cost tables, 120 cells except the 62 fair cells of D – A.

tuned thresholds directly on the entropy score, improves on A-hard by +1.44 points and is statistically indistinguishable from D (−0.03 points, CI −0.11 to +0.09) at about 1% lower cost. The advantage over rank escalation therefore comes from choosing the operating point under the hard budget on the fit half, and with a single scalar probe a monotone threshold rule already captures the information that the four calibrators add.

Robustness. Replacing the median cost table by the p95 or p99 table, for both the costs and the budget grid, leaves the conclusions unchanged: D still selects no route whose

tabulated cost exceeds the budget, and its gains over A and A-hard change by at most 0.04 points. Calibrators fitted without the evaluated condition (LOCO) also keep D free of violations and ahead of A-hard by +1.59 points, so the gain does not depend on having calibrated on the test condition itself. On the held-out halves, the four candidate-specific calibrators have mean absolute errors of 0.019–0.057 in per-image pixel error, lie close to the diagonal of a reliability plot, and change by at most 0.003 when the ten equal-count bins are replaced by five bins or isotonic regression (supplement, Fig. S5). The equivalence of D and T-hard is therefore not caused by poor calibration.

Against static deployment. The comparison with static deployment depends on what the static model is charged. If it is charged the cost of the route that ends in it, as if it also paid for the probe, D stays within 0.16 points of the best feasible static candidate (95% CI −0.22 to −0.10), tying in 92 of 120 cells and lowering the mean route cost by 2.0%; the non-deployable GT pixel-error oracle exceeds that static reference by only 0.11 points, so the headroom for per-image routing under a per-frame cap is small with this candidate set. A static deployment, however, does not run the probe. Figure 8 shows the same-harness latency of each capacity deployed statically and reached through the tiny probe. On E3 a route costs 0.7 ms more than the static engine it ends in for tiny and 2.7 ms more for large; on E1 the overhead is 38–47 ms, comparable to the full latency of the large engine, and it does not shrink with the HailoRT scheduler or UINT8 output streams. Charged its own latency, the best feasible static model beats D by 2.62 points on E3 (0 wins, 61 ties, 59 losses) and by 7.41 (FLOAT32) to 9.15 (UINT8) points on E1 (0 wins); T-hard behaves identically. At the lowest budgets of each device

Table 8: Condition-level comparison against policy A (median cost tables). W/T/L and mean Δ use fair cells; violation counts use all 24 cells per condition. Configured and pooled D each have zero violations.

Condition	Fair	Config. W/T/L	Config. Δ	Pooled W/T/L	Pooled Δ	A viol.	D viol.
Cityscapes	6/24	6/0/0	+8.81	6/0/0	+8.87	18/24	0/24
ACDC/Fog	9/24	9/0/0	+5.38	9/0/0	+5.51	15/24	0/24
ACDC/Night	14/24	14/0/0	+1.05	14/0/0	+1.08	10/24	0/24
ACDC/Rain	21/24	15/6/0	+2.09	15/6/0	+2.11	3/24	0/24
ACDC/Snow	12/24	10/0/2	+1.49	10/0/2	+1.48	12/24	0/24

no route is feasible at all, because the cheapest route, tiny plus probe, already exceeds the static tiny engine.

Mean-cost budgets and break-even overhead. When only the average cost is constrained, the fairer static reference is a random mixture of the two adjacent static models with the same mean cost. Charged their own latencies, candidate-specific routing still loses to this mixture by 0.3–0.5 points on E3 and 5.5–6.3 points on E1. To ask how much the overhead would have to shrink, we scale the measured per-capacity overhead by $\alpha \in [0, 1]$ and repeat the comparison (Fig. 9). With zero overhead, routing would gain 1.3–1.4 points over static mixing on both devices; the gain vanishes at 1.3 ms of overhead on E3 (measured: 1.9 ms) and 4.3 ms on E1 (measured: 44 ms). Under per-frame budgets routing never wins, even at zero overhead (−0.16 points), because any per-frame cap forces the router to stay at or below the capacity that the static model already uses.

Table 8 breaks the comparison with A down by condition, with 24 cells per row from three runs and two cost tables. The gain over A is largest on clean Cityscapes (+8.81 points), although only 6 of its 24 cells are fair because A violates the budget in the other 18. Rain is the opposite case: A violates a budget in only 3 cells, and configured D records 15 wins, 6 ties, and no losses over 21 fair cells with a +2.09-point gain. Night has the smallest gain, +1.05 points, despite winning all 14 fair cells. The only two losses occur on snow, for both configured and pooled D. No single condition drives the aggregate result, and fair-cell counts and violation counts must be read together.

The small night gain is consistent with the probe. The large candidate reaches 59.0 mIoU on Cityscapes but only 38.2 on ACDC night, and the night calibrators have the largest held-out errors (0.051–0.057) and the weakest score–error rank correlation (Spearman 0.35–0.67, against 0.74–0.81 on Cityscapes). The entropy score thus ranks errors least reliably exactly where errors are most frequent. Figure 10 shows one representative held-out image per capacity that policy D selects.

5.4. Compiled-engine accuracy and on-device replay

The replay above uses PyTorch predictions. Compiled TensorRT FP16 engines with the trained weights reproduce PyTorch within ± 0.02 points on every capacity and split, with 99.7–100% pixel agreement and a probe-entropy error below 2×10^{-4} . Hailo-8 INT8 engines lose up to 1.6

points on tiny (pixel agreement down to 87% at night) and at most 0.5 points on large; the probe entropy deviates by 0.012–0.035 on average but keeps a rank correlation of at least 0.88. Replaying Run A with the on-device predictions and probe scores leaves every conclusion unchanged: on E3 D differs from the PyTorch replay by 0.01 points, and on E1 D loses a further 0.6 points but still matches T-hard (−0.06) and trails static deployment by 7.9 points.

6. Discussion

6.1. When adaptive routing pays off

The comparison with static deployment is the central result. Routing can only pay off if choosing per image is worth more than its cost. The headroom is small: the GT pixel-error oracle exceeds the best feasible static candidate by only 0.11 points under a per-frame cap, and even at zero overhead routing gains only 1.3–1.4 points over static mixing under a mean-cost budget. Against that, every route pays for the tiny engine, the entropy and decision, and on the Hailo-8 an engine switch that dominates the cost. Routing therefore becomes worthwhile only when three conditions hold: the budget constrains mean rather than per-frame cost; the overhead stays below the break-even value (about 1.3 ms on E3 and 4.3 ms on E1 here); and the candidates differ enough per image to leave headroom. For a fixed per-frame deadline, a static model of the largest feasible capacity is the stronger choice and should always be reported as a baseline, timed in the same harness as the adaptive pipeline.

6.2. What drives the gain among routers

Policies A and D read the same probe score, yet they diverge at tight budgets. A maps the score to a latency rank and escalates whenever the score crosses its threshold; nothing in that rule prevents escalation to a route whose measured cost exceeds the budget, so A’s operating point must be chosen conservatively, or it violates the budget, as it does in 18 of 24 Cityscapes cells. Imposing the budget alone repairs the violations but improves fair-cell quality by only +0.47 points, whereas the remaining +1.41 points come from choosing, within the feasible set, an operating point tuned on the fit half. T-hard obtains the same gain with three thresholds on the raw score, so with one scalar probe the four candidate-specific calibrators are an interpretable but not a necessary way to make that choice. A richer probe, for example per-class or spatial uncertainty, would be needed for candidate-specific calibration to carry information that a threshold rule cannot.

6.3. Deployment guidance

For a new accelerator, the results suggest a simple procedure: compile the fixed candidate set, measure complete routes and static deployments with the production runtime in the same harness, preferably reporting tail percentiles as well as the median, and compare the routing overhead with

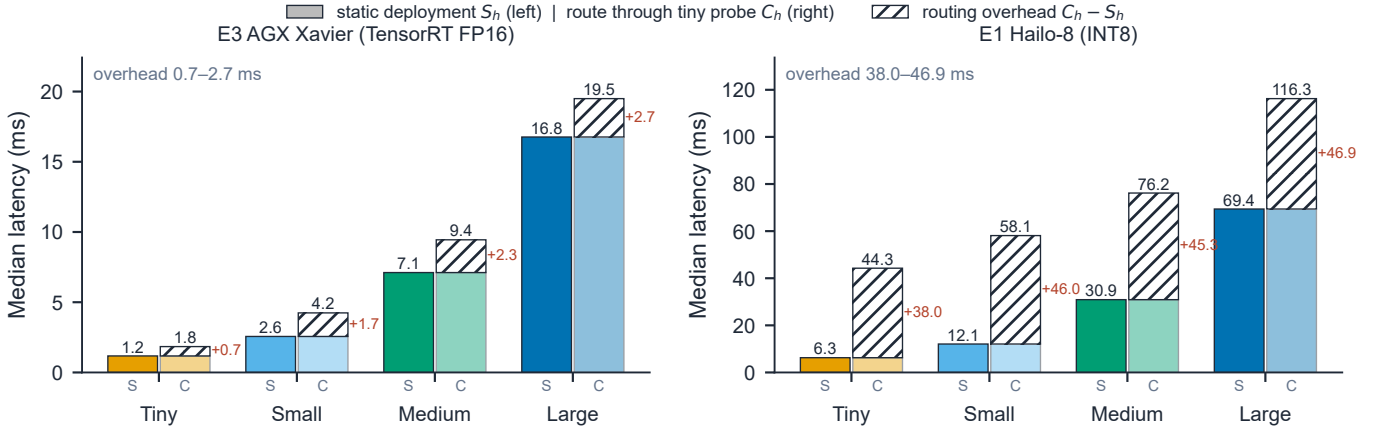


Figure 8: The routing overhead is small on the TensorRT GPU but comparable to a full large-model inference on the Hailo-8. Same-harness median latency of each capacity deployed statically (S_h , left bars) and reached through the tiny probe (C_h , right bars); the hatched segment is the overhead $C_h - S_h$, labelled in ms. E3 returns logits; E1 uses explicit network-group activation with FLOAT32 output streams.

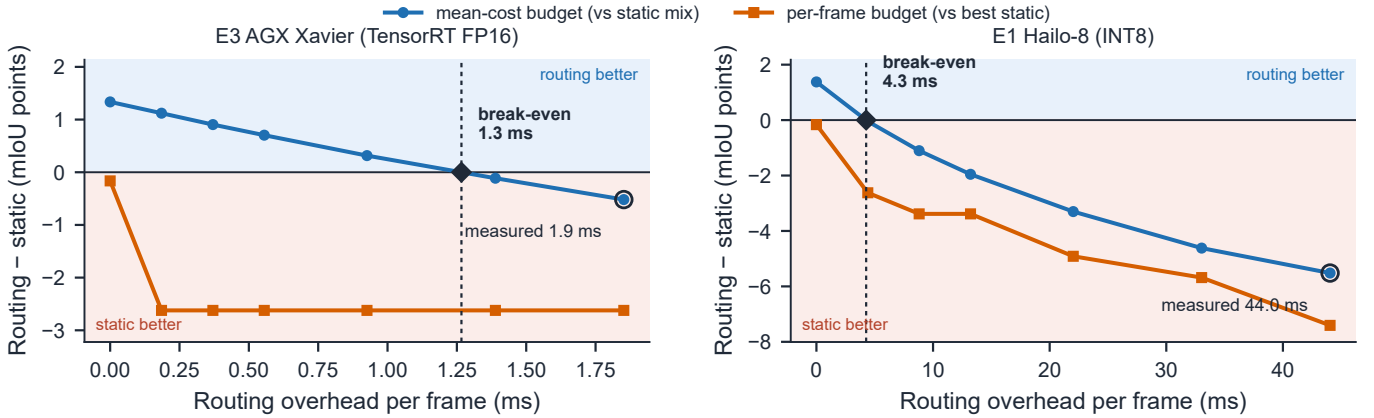


Figure 9: Routing pays off only under mean-cost budgets and only below a small break-even overhead. Routing minus static mIoU as the measured routing overhead is scaled from zero to its measured value (circle); shading marks where routing (blue) or static deployment (red) is better. Blue line: mean-cost budgets against static mixing; orange line: per-frame budgets against the best feasible static model, which stays below zero at every overhead. Diamonds: break-even overhead.

the break-even value before deploying a router. Designs that reuse the probe computation inside the selected candidate, keep all candidates resident without switching (multi-context compilation on Hailo, one engine with shared early layers on TensorRT), or route among models with more complementary errors attack exactly the terms that make routing lose here. If a router is used, pooled calibration produced the same result as configured calibration (54/6/2 against A) and, in the LOCO analysis, transferred to a condition it had not seen, so it is the simpler default; a threshold router is simpler still. Because the probe is the tiny candidate, its quality bounds the information available to the router; the smallest gains occurred at night, where the probe ranks errors worst.

6.4. Relation to latency predictors and dynamic networks

The study does not compare against learned latency predictors [39], and RQ1 is not evidence against them.

With four candidates, measuring every route is cheap and exact; with a large search space such as Once-for-All [2], measurement becomes impractical and a predictor is needed, at the cost of predictor error in the feasibility test. Likewise, PACE-Seg gives up the fine granularity of in-graph dynamic networks for compatibility with static compilers. In-graph methods avoid the engine switch that dominates the Hailo-8 overhead, but they also need an accelerator that supports data-dependent control flow; whether a larger static candidate set would increase per-image headroom without weakening calibration remains open.

6.5. Limitations and threats to validity

Data. Calibration is evaluated on Cityscapes and four ACDC conditions. The LOCO analysis withholds one condition at a time, but all five conditions come from the

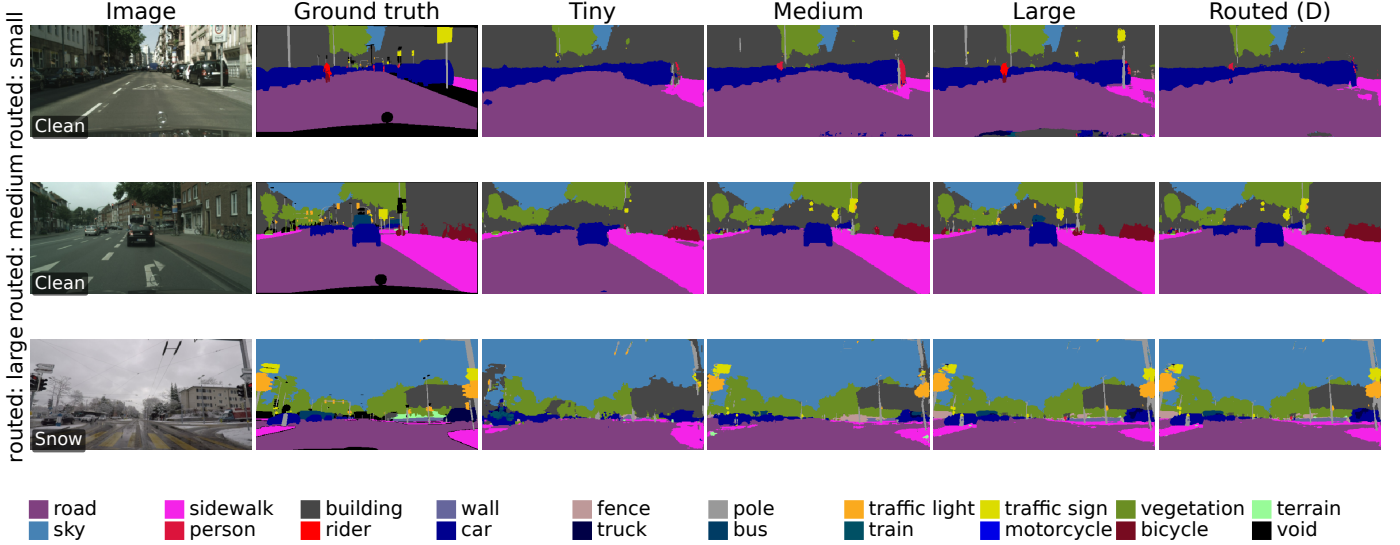


Figure 10: Input adaptivity on held-out images. Selection rule fixed before inspection: policy D of Run A on E3 at the budget of the large route, pooled over the five held-out splits; for each capacity that D selects, the image whose selected-capacity pixel error is closest to the median of that route class. D selected tiny for none of the 453 images at this budget, small for 27, medium for 61, and large for 365. Columns show the input, ground truth (void in black), the tiny, medium, and large candidates, and the routed output.

same two datasets and camera setups, so transfer to new sensors or cities is untested. The fit and held-out halves alternate by image index rather than by sequence or location, so correlated neighbouring frames may fall on both sides; the alternating halves are not an external test set. Fit halves of 50–250 images per split leave about five images per calibration bin on the smaller ACDC splits, although five bins and isotonic regression gave nearly identical held-out errors. The calibration target is per-image pixel error, which agrees only moderately with per-image 1–mIoU (Spearman 0.23–0.58 across splits); routing on a class-balanced target may favour rare classes more.

Statistics. The 120 operating cells reuse images, predictions, and thresholds and are not independent samples. The image-level bootstrap intervals capture image sampling variation but not training-run variation; the nested bootstrap adds fit-half resampling, and three seeds per model are too few to estimate seed variance precisely.

Measurement. Accuracy under routing is replayed offline from recorded predictions with measured costs rather than in a closed-loop deployment, and compiled-engine accuracy and the on-device replay cover one of the three supernet. The zero-violation result is a route-selection guarantee with respect to tabulated median, p95, or p99 costs, not a measured per-frame deadline guarantee, and it is partly structural.

Scope. The study has four candidates, one entropy probe, two complete-route backends, and no energy measurement. Cold-start behaviour, temporal routing across video frames, and class-level error analysis are left to future work.

7. Conclusion

We measured when input-adaptive semantic segmentation pays off on edge accelerators. On four edge devices, a FLOPs-proportional proxy did not reproduce measured budget choices, so feasibility should come from measured complete inference routes. Shared elastic training cost the largest capacity 0.9 mIoU points on Cityscapes and 1.6–2.8 points on ACDC relative to training it alone. Among routers, a hard measured budget with fit-half operating-point tuning is what matters: it lifted routing 2.87 points over rank escalation, its selected routes always fit the tabulated median, p95, and p99 costs, and a three-threshold scalar router matched candidate-specific calibration. Charged its own measured latency, however, static deployment beat routing on both a TensorRT GPU and a Hailo-8, and routing would break even under mean-cost budgets only if its overhead fell below about 1.3 and 4.3 ms. Adaptive-inference studies on edge hardware should therefore report a static baseline timed in the same harness and the break-even overhead of their routing pipeline. Offline replay, an index-based split, and the absence of energy measurement bound these claims.

CRedit authorship contribution statement

Dng Nguyn: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Visualization, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could

have appeared to influence the work reported in this paper.

Funding

[FUNDING STATEMENT TO BE PROVIDED.]

Data availability

The implementation, experiment reports, canonical result artifacts, and a separate supplement containing detailed provenance, runtime versions, and extended qualitative and routing analyses are recorded in the project repository [REPOSITORY URL OR DOI]. Cityscapes and ACDC are publicly available from their providers and are subject to their respective licenses; they are not redistributed.

Acknowledgements

[ACKNOWLEDGEMENTS TO BE PROVIDED.]

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author used Claude (Anthropic) in order to assist with language editing, manuscript structuring, LaTeX formatting, analysis scripts, and scripts that render figures from the recorded experimental results. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

References

- [1] J. Yu, T. Huang, Universally slimmable networks and improved training techniques, in: IEEE/CVF International Conference on Computer Vision, 2019, pp. 1803–1811. doi:10.1109/ICCV.2019.00189.
- [2] H. Cai, C. Gan, T. Wang, Z. Zhang, S. Han, Once-for-All: Train one network and specialize it for efficient deployment, in: International Conference on Learning Representations, 2020. URL <https://openreview.net/forum?id=HylxE1HKwS>
- [3] J. Long, E. Shelhamer, T. Darrell, Fully convolutional networks for semantic segmentation, in: IEEE Conference on Computer Vision and Pattern Recognition, 2015, pp. 3431–3440.
- [4] H. Zhao, J. Shi, X. Qi, X. Wang, J. Jia, Pyramid scene parsing network, in: IEEE Conference on Computer Vision and Pattern Recognition, 2017, pp. 2881–2890.
- [5] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, H. Adam, Encoder-decoder with atrous separable convolution for semantic image segmentation, in: European Conference on Computer Vision, 2018, pp. 801–818.
- [6] A. Paszke, A. Chaurasia, S. Kim, E. Culurciello, ENet: A deep neural network architecture for real-time semantic segmentation, arXiv preprint arXiv:1606.02147 (2016).
- [7] E. Romera, J. Alvarez, L. Bergasa, R. Arroyo, ERFNet: Efficient residual factorized ConvNet for real-time semantic segmentation, IEEE Transactions on Intelligent Transportation Systems 19 (1) (2018) 263–272.
- [8] H. Zhao, X. Qi, X. Shen, J. Shi, J. Jia, ICNet for real-time semantic segmentation on high-resolution images, in: European Conference on Computer Vision, 2018, pp. 405–420.
- [9] R. Poudel, S. Liwicki, R. Cipolla, Fast-SCNN: Fast semantic segmentation network, in: British Machine Vision Conference, 2019.
- [10] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, L.-C. Chen, MobileNetV2: Inverted residuals and linear bottlenecks, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2018, pp. 4510–4520.
- [11] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. Le, H. Adam, Searching for MobileNetV3, in: IEEE/CVF International Conference on Computer Vision, 2019, pp. 1314–1324.
- [12] C. Yu, J. Wang, C. Peng, C. Gao, G. Yu, N. Sang, BiSeNet: Bilateral segmentation network for real-time semantic segmentation, in: European Conference on Computer Vision, 2018, pp. 325–341. URL https://openaccess.thecvf.com/content_ECCV_2018/html/Changqian_Yu_BiSeNet_Bilateral_Segmentation_ECCV_2018_paper.html
- [13] C. Yu, C. Gao, J. Wang, G. Yu, C. Shen, N. Sang, BiSeNet V2: Bilateral network with guided aggregation for real-time semantic segmentation, International Journal of Computer Vision 129 (2021) 3051–3068.
- [14] M. Fan, S. Lai, J. Huang, X. Wei, Z. Chai, J. Luo, X. Wei, Rethinking BiSeNet for real-time semantic segmentation, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2021, pp. 9716–9725. URL https://openaccess.thecvf.com/content/CVPR2021/html/Fan_Rethinking_BiSeNet_for_Real-Time_Semantic_Segmentation_CVPR_2021_paper.html
- [15] H. Pan, Y. Hong, W. Sun, Y. Jia, Deep dual-resolution networks for real-time and accurate semantic segmentation of traffic scenes, IEEE Transactions on Intelligent Transportation Systems 24 (3) (2023) 3448–3460.
- [16] J. Xu, Z. Xiong, S. Bhattacharyya, PIDNet: A real-time semantic segmentation network inspired by PID controllers, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023, pp. 19529–19539. URL https://openaccess.thecvf.com/content/CVPR2023/html/Xu_PIDNet_A_Real-Time_Semantic_Segmentation_Network_Inspired_by_PID_Controllers_CVPR_2023_paper.html
- [17] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. Alvarez, P. Luo, SegFormer: Simple and efficient design for semantic segmentation with transformers, Advances in Neural Information Processing Systems 34 (2021) 12077–12090.
- [18] S.-C. Zhang, Y. Li, Y.-H. Wu, Q. Hou, M.-M. Cheng, Revisiting efficient semantic segmentation: Learning offsets for better spatial and class feature alignment, in: IEEE/CVF International Conference on Computer Vision, 2025, pp. 22361–22371. URL https://openaccess.thecvf.com/content/ICCV2025/html/Zhang_Revisiting_Efficient_Semantic_Segmentation_Learning_Offsets_for_Better_Spatial_and_ICCV_2025_paper.html
- [19] J. Yu, L. Yang, N. Xu, J. Yang, T. Huang, Slimmable neural networks, in: ICLR Workshop, 2019. URL <https://arxiv.org/abs/1812.08928>
- [20] D. Xue, F. Yang, P. Wang, L. Herranz, J. Sun, Y. Zhu, Y. Zhang, SlimSeg: Slimmable semantic segmentation with boundary supervision, in: Proceedings of the 30th ACM International Conference on Multimedia, 2022, pp. 6539–6548. doi:10.1145/3503161.3548191.
- [21] J. Haberer, J. E. Wilhelm, O. Landsiedel, Slimmable ConvNeXt: Width-adaptive inference for efficient multi-device deployment, arXiv:2605.22677 (2026). arXiv:2605.22677.
- [22] S. Teerapittayanon, B. McDanel, H. Kung, BranchyNet: Fast inference via early exiting from deep neural networks, in: International Conference on Pattern Recognition, 2016, pp. 2464–2469.
- [23] G. Huang, D. Chen, T. Li, F. Wu, L. van der Maaten, K. Weinberger, Multi-scale dense networks for resource efficient image classification, in: International Conference on Learning Representations, 2018.

- [24] X. Wang, F. Yu, Z.-Y. Dou, T. Darrell, J. Gonzalez, SkipNet: Learning dynamic routing in convolutional networks, in: European Conference on Computer Vision, 2018, pp. 409–424.
- [25] Z. Wu, T. Nagarajan, A. Kumar, S. Rennie, L. Davis, K. Grauman, R. Feris, BlockDrop: Dynamic inference paths in residual networks, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2018, pp. 8817–8826.
- [26] Y. Han, G. Huang, S. Song, L. Yang, H. Wang, Y. Wang, Dynamic neural networks: A survey, IEEE Transactions on Pattern Analysis and Machine Intelligence 44 (11) (2022) 7436–7456.
- [27] Y. Li, L. Song, Y. Chen, Z. Li, X. Zhang, X. Wang, J. Sun, Learning dynamic routing for semantic segmentation, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 8553–8562.
- [28] Z. Liu, Z. Xu, H.-J. Wang, T. Darrell, E. Shelhamer, Anytime dense prediction with confidence adaptivity, in: International Conference on Learning Representations, 2022. URL <https://openreview.net/forum?id=kNKF0XleuC>
- [29] A. Kouris, S. Venieris, S. Laskaridis, N. Lane, Multi-exit semantic segmentation networks, in: European Conference on Computer Vision, 2022, pp. 330–349.
- [30] M. Tan, B. Chen, R. Pang, V. Vasudevan, M. Sandler, A. Howard, Q. Le, MnasNet: Platform-aware neural architecture search for mobile, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 2820–2828.
- [31] H. Cai, L. Zhu, S. Han, ProxylessNAS: Direct neural architecture search on target task and hardware, in: International Conference on Learning Representations, 2019.
- [32] B. Wu, X. Dai, P. Zhang, Y. Wang, F. Sun, Y. Wu, Y. Tian, P. Vajda, Y. Jia, K. Keutzer, FBNet: Hardware-aware efficient ConvNet design via differentiable neural architecture search, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 10734–10742.
- [33] W. Chen, X. Gong, X. Liu, Q. Zhang, Y. Li, Z. Wang, FasterSeg: Searching for faster real-time semantic segmentation, in: International Conference on Learning Representations, 2020. URL <https://openreview.net/forum?id=BJgqQ6NYvB>
- [34] Z. Dou, D.-Y. Ye, B. Wang, X. Wang, S. Sun, AutoSegEdge: Searching for the edge device real-time semantic segmentation based on multi-task learning, Image and Vision Computing 136 (2023). doi:10.1016/j.imavis.2023.104719.
- [35] Z. Dou, D.-Y. Ye, B. Wang, X. Wang, S. Sun, Multi-objective neural architecture search for efficient and fast semantic segmentation on edge, IEEE Transactions on Intelligent Vehicles 9 (1) (2024) 1346–1357. doi:10.1109/TIV.2023.3332594.
- [36] Y. Li, C. Yang, P. Zhao, G. Yuan, W. Niu, J. Guan, H. Tang, M. Qin, Q. Jin, B. Ren, X. Lin, Y. Wang, Towards real-time segmentation on the edge, Proceedings of the AAAI Conference on Artificial Intelligence 37 (2) (2023) 1339–1347. doi:10.1609/aaai.v37i2.25232.
- [37] Y. Kwon, W. Kim, H. Kim, HARD: Hardware-aware lightweight real-time semantic segmentation model deployable from edge to GPU, in: Asian Conference on Computer Vision, 2024, pp. 3552–3569. URL https://openaccess.thecvf.com/content/ACCV2024/html/Kwon_HARD_Hardware-Aware_lightweight_Real-time_semantic_segmentation_model_Deployable_from_ACCV_2024_paper.html
- [38] C. Li, Z. Yu, Y. Fu, Y. Zhang, Y. Zhao, H. You, Q. Yu, Y. Wang, C. Hao, Y. Lin, HW-NAS-Bench: Hardware-aware neural architecture search benchmark, in: International Conference on Learning Representations, 2021.
- [39] L. Zhang, S. Han, J. Wei, N. Zheng, T. Cao, Y. Yang, Y. Liu, nn-Meter: Towards accurate latency prediction of deep-learning model inference on diverse edge devices, in: ACM International Conference on Mobile Systems, Applications, and Services, 2021, pp. 81–93.
- [40] Y. Gal, Z. Ghahramani, Dropout as a Bayesian approximation: Representing model uncertainty in deep learning, in: International Conference on Machine Learning, 2016, pp. 1050–1059.
- [41] B. Lakshminarayanan, A. Pritzel, C. Blundell, Simple and scalable predictive uncertainty estimation using deep ensembles, Advances in Neural Information Processing Systems 30 (2017).
- [42] A. Kendall, Y. Gal, What uncertainties do we need in Bayesian deep learning for computer vision?, Advances in Neural Information Processing Systems 30 (2017).
- [43] C. Guo, G. Pleiss, Y. Sun, K. Weinberger, On calibration of modern neural networks, in: International Conference on Machine Learning, 2017, pp. 1321–1330.
- [44] B. Zadrozny, C. Elkan, Transforming classifier scores into accurate multiclass probability estimates, in: ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2002, pp. 694–699.
- [45] Y. Geifman, R. El-Yaniv, Selective classification for deep neural networks, Advances in Neural Information Processing Systems 30 (2017).
- [46] A. Angelopoulos, S. Bates, A. Fisch, L. Lei, T. Schuster, Conformal risk control, in: International Conference on Learning Representations, 2024.
- [47] D. Wang, B. Gong, L. Wang, On calibrating semantic segmentation models: Analyses and an algorithm, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023, pp. 23652–23662.
- [48] P. de Jorje, R. Volpi, P. Torr, G. Rogez, Reliability in semantic segmentation: Are we on the right track?, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023, pp. 2456–2465.
- [49] C. Sakaridis, D. Dai, L. V. Gool, Semantic foggy scene understanding with synthetic data, International Journal of Computer Vision 126 (2018) 973–992.
- [50] C. Sakaridis, D. Dai, L. V. Gool, Guided curriculum model adaptation and uncertainty-aware evaluation for semantic nighttime image segmentation, in: IEEE/CVF International Conference on Computer Vision, 2019, pp. 7374–7383.
- [51] C. Sakaridis, D. Dai, L. V. Gool, ACDC: The adverse conditions dataset with correspondences for semantic driving scene understanding, in: IEEE/CVF International Conference on Computer Vision, 2021, pp. 10765–10775.
- [52] G. Hinton, O. Vinyals, J. Dean, Distilling the knowledge in a neural network, arXiv preprint arXiv:1503.02531 (2015).
- [53] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, B. Schiele, The Cityscapes dataset for semantic urban scene understanding, in: IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 3213–3223.